



ROUTEZONE

Prédiction de la gravité des accidents routiers

RAPPORT PROFESSIONNEL — ÉPREUVE E1

Collecte, préparation et exposition des données

Candidate	Meriem Abdelouahed
Formation	Développeur en Intelligence Artificielle — Simplon × Microsoft
Certification	RNCP37827
Soutenance	Juin 2026 — distanciel
Bloc / Compétences	Bloc 1 — Compétences C1 à C5
Sources de données	BAAC 2022-2024 (data.gouv.fr) — Open-Meteo — ONISR — OSRM — SAU & casernes

Sommaire

1. Contexte et objectif.....	2
2. Collecte des données.....	2
3. Nettoyage et préparation.....	3
4. Base de données PostgreSQL.....	4
5. API REST Data — FastAPI.....	4
6. Conclusion.....	4

1. Contexte et objectif

RouteZone est un système de machine learning visant à prédire la gravité d'un accident routier (Grave : tué ou hospitalisé / Pas grave : indemne ou blessé léger) à partir des données BAAC (Bulletin d'Analyse des Accidents Corporels) 2022-2024, publiées en open data par le Ministère de l'Intérieur.

Le projet répond à une problématique de sécurité routière : aider les autorités et les services de secours à anticiper les cas critiques pour mieux prioriser les interventions. La spécificité de RouteZone réside dans l'intégration du concept de Golden Hour, principe médical établi en traumatologie selon lequel la prise en charge d'une victime grave dans les 60 minutes suivant le traumatisme augmente significativement les chances de survie.

Périmètre du Bloc E1

Le Bloc E1 couvre la collecte, la préparation et l'exposition des données. Il constitue le socle qui alimente l'ensemble du projet.

Compétence	Action réalisée	Livrable
C1 — Automatiser l'extraction	12 CSV BAAC + API Open-Meteo + scraping ONISR + enrichissement OSRM	4 notebooks + 3 scripts .py
C2 — Sources fiables	Identification et validation des sources open data officielles	Documentation README
C3 — Nettoyage / transformation	Gestion des NaN, des types, des colonnes inutiles, merge des 4 tables	notebook_01 + dataset_clean.csv
C4 — Base de données	Modélisation Merise (MCD/MPD), PostgreSQL 16, scripts d'import	schema_merise.md + create_db.sql
C5 — API REST Data	Développement FastAPI 7 endpoints documentés Swagger	src/api/routes_data.py

2. Collecte des données

La collecte s'appuie sur quatre sources complémentaires : le jeu de données principal BAAC, deux enrichissements externes (Open-Meteo et ONISR), et un enrichissement calculé via OSRM.

2.1 Données BAAC

Les fichiers BAAC sont organisés en 4 tables liées par la clé Num_Acc : caract (caractéristiques de l'accident), lieux (type de route, vitesse, surface), usagers (âge, sexe, gravité, équipement), vehicules (catégorie, type de collision). Au total, 12 fichiers CSV ont été chargés, représentant 153 054 accidents sur 3 ans (2022-2024), impliquant 413 570 usagers et environ 270 000 véhicules.

2.2 Enrichissement Open-Meteo

La colonne atm du BAAC code les conditions météo de façon déclarative et subjective. L'API Open-Meteo (gratuite, sans clé) fournit la météo réelle heure par heure par coordonnées GPS. L'enrichissement ajoute 4 variables au dataset (temperature, precipitation, windspeed, weathercode). La collecte s'effectue via une boucle Python avec une pause de 0,1 seconde entre chaque requête pour respecter les limites de l'API.

2.3 Scraping ONISR

Le baromètre mensuel ONISR (nombre de tués par mois) n'est pas disponible en CSV. Il est scrapé via requests et BeautifulSoup. Lors de l'inspection du HTML, j'ai découvert un paramètre d'URL field_annees_target_id permettant de cibler directement une année spécifique sans parcourir toutes les pages, ce qui réduit le nombre de requêtes. Les données scrapées servent ensuite à valider la qualité du dataset BAAC en comparant les comptages mensuels.

2.4 Enrichissement OSRM — Golden Hour

Le Golden Hour est un principe médical fondamental : la prise en charge d'une victime grave dans les 60 minutes maximise les chances de survie. Or, le BAAC ne contient aucune information sur le temps d'intervention des secours. J'ai donc décidé d'intégrer cette dimension comme variable explicative.

OSRM (Open Source Routing Machine) est un moteur de routage haute performance qui calcule des itinéraires réels sur le réseau routier d'OpenStreetMap, plutôt qu'une approximation à vol d'oiseau (formule

de Haversine). Le pipeline calcule, pour chaque accident, le temps de trajet depuis la caserne de pompiers la plus proche puis vers le Service d'Accueil des Urgences (SAU) le plus proche. Les données externes utilisées proviennent de data.gouv.fr : 5 906 casernes de pompiers (CIS) et 638 SAU.

La comparaison entre l'approximation Haversine et le calcul OSRM révèle des résultats marquants : OSRM détecte 47 % de zones blanches supplémentaires (temps de prise en charge supérieur à 30 minutes). La corrélation entre temps d'intervention et taux de décès est statistiquement renforcée, et cette feature OSRM devient le facteur le plus discriminant du modèle final.

3. Nettoyage et préparation

Le nettoyage du dataset s'est effectué en 10 étapes ordonnées sur les tables brutes avant la fusion finale. La méthodologie suit le principe : nettoyer chaque table indépendamment, puis fusionner, pour limiter la propagation d'erreurs.

3.1 Étapes principales

- Correction du nom de colonne Accident_Id en Num_Acc sur caract 2022 (bug de nommage qui aurait cassé les merges)
- Concaténation des 3 années pour chaque table, remplacement des -1 par NaN (convention BAAC : -1 = non renseigné)
- Visualisation des NaN avec missingno, suppression des doublons stricts détectés dans lieux
- Gestion des types : conversion des colonnes numériques lues comme texte par pandas
- Correction des coordonnées GPS (virgule à point) et filtrage des DOM-TOM (9 578 lignes retirées)
- Gestion des colonnes inutiles : suppression de celles ayant plus de 90 % de NaN ou sans intérêt pour la modélisation
- Gestion des NaN : suppression pour les colonnes critiques (grav, sexe, dep), imputation par le mode pour les variables catégorielles, par la médiane pour les variables numériques
- Vérification finale : 0 NaN restant dans toutes les tables nettoyées

La fusion des 4 tables s'effectue via des left joins sur Num_Acc. Trois variables dérivées ont été créées : age (calculé à partir de an - an_nais), heure (extraite de hrmn), jour_semaine (dérivé de la date complète).

3.2 Analyses de biais

Deux analyses de biais ont été réalisées pour assurer une lecture critique des données : biais de représentation par sexe (volume brut vs taux corrigé par exposition) et biais géographique (volume brut par département vs taux pour 100 000 habitants). Ces analyses illustrent qu'un volume élevé n'est pas synonyme de risque élevé sans normalisation.

4. Base de données PostgreSQL

La base RouteZone est modélisée selon la méthode Merise (MCD puis MPD) et implémentée en PostgreSQL 16. PostgreSQL a été choisi pour sa conformité ACID (intégrité des transactions), son support natif des types géospatiaux (utile pour les coordonnées GPS), sa gestion de la concurrence (accès simultané par l'API, les scripts d'import et le monitoring), et son excellente intégration avec Docker et l'écosystème Python via SQLAlchemy.

La base contient 8 tables organisées en deux groupes : les données métier (accidents, lieux, usagers, vehicules, meteo, barometre_onisr) et les données applicatives (users avec mot de passe hashé en bcrypt, predictions avec historique daté).

Les scripts d'alimentation sont import_data.py (153 054 accidents importés), collect_meteo.py (enrichissement Open-Meteo) et scraping_onisr.py (baromètres mensuels).

5. API REST Data — FastAPI

L'API expose les données collectées via des endpoints REST documentés automatiquement par Swagger UI. Elle est construite avec FastAPI (framework Python moderne basé sur Pydantic) et lancée par la commande `python -m uvicorn main:app --reload`.

L'API intègre une authentification par clé API (header X-API-Key) pour les endpoints métier, une validation automatique des données entrantes via Pydantic, et une documentation interactive sur /docs (Swagger) et /redoc.

Endpoint	Description
GET /	Health check (vérifie que l'API est en ligne)
GET /accidents	Liste filtrable par département, année, gravité
GET /accidents/stats	Statistiques globales (total, répartition, top départements)
GET /accidents/departement/{dep}	Statistiques mensuelles pour un département
GET /accidents/gravite	Répartition des usagers par niveau de gravité
GET /meteo/stats	Statistiques météo agrégées
GET /onisr/barometre	Baromètres mensuels ONISR avec filtre par année

6. Conclusion

Le Bloc E1 constitue le socle de données du projet RouteZone. Les livrables produits dans cette phase sont :

- Dataset nettoyé : 153 054 accidents, 413 570 usagers, 0 NaN (dataset_clean.csv)
- Dataset enrichi : ajout de temperature, precipitation, windspeed, weathercode, temps_total_osrm
- Base de données PostgreSQL 16 : 8 tables, schéma Merise documenté
- 4 notebooks documentés (exploration, météo, ONISR, OSRM) et 4 scripts de production
- API FastAPI : 7 endpoints documentés avec Swagger automatique

La spécificité méthodologique de ce bloc — l'intégration du Golden Hour via OSRM — constitue l'apport différenciant par rapport aux approches classiques sur ce dataset. Cette feature dérivée est devenue le facteur le plus discriminant du modèle final, validant l'hypothèse de départ selon laquelle le temps d'intervention des secours est un facteur clé de la gravité finale d'un accident.

Le dataset enrichi et l'API de données alimenteront le Bloc E3 : entraînement et comparaison de modèles de classification pour prédire la gravité des accidents.